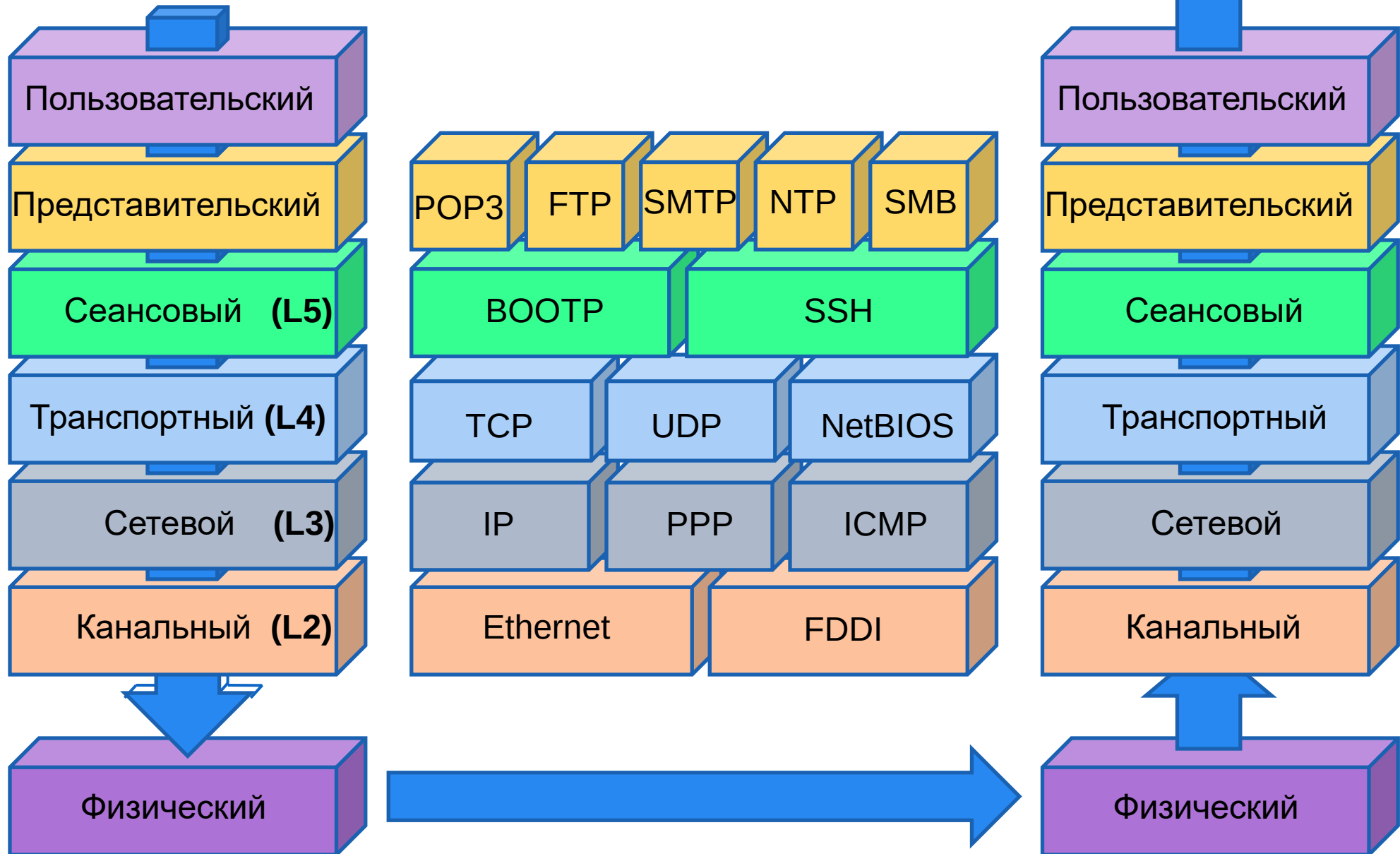


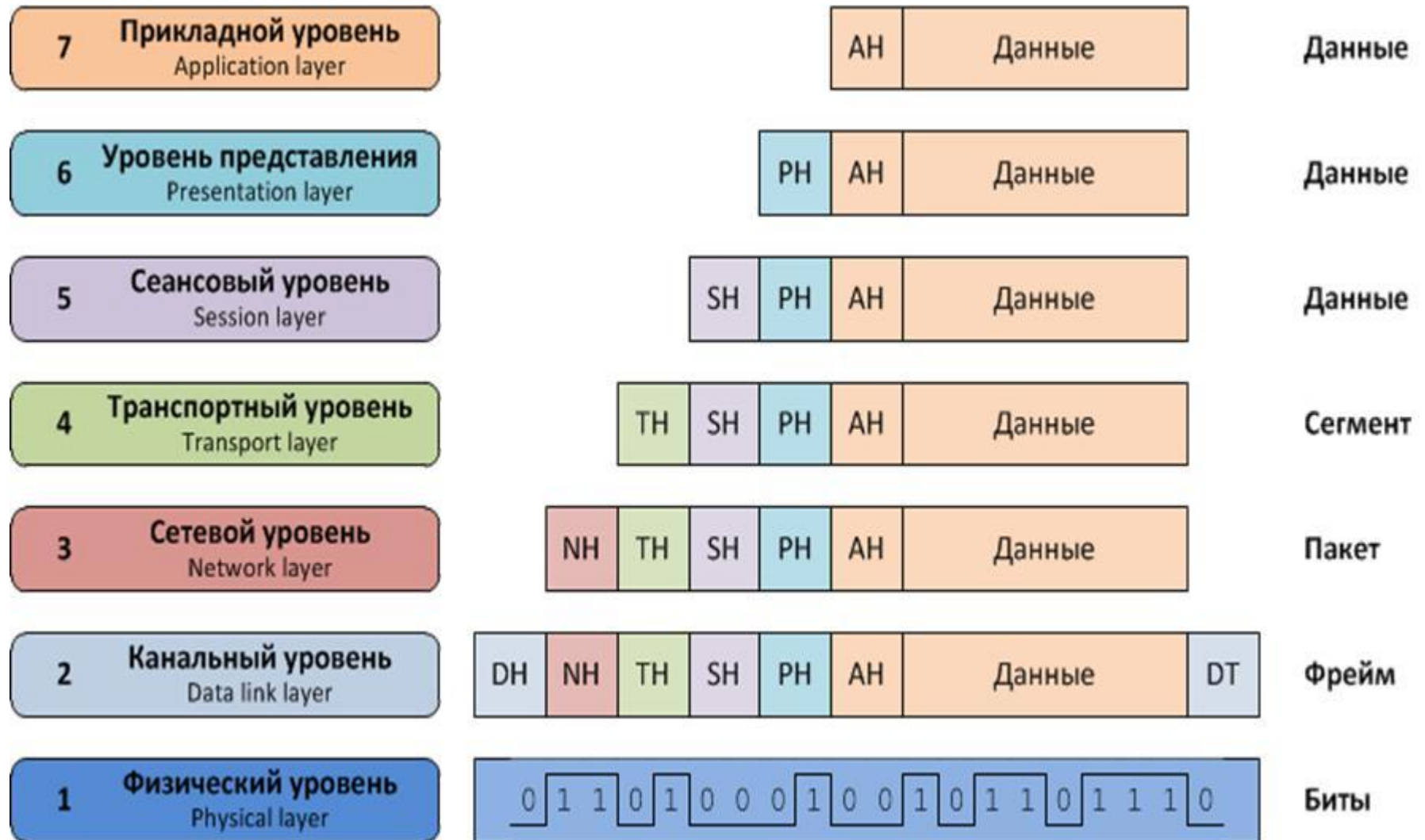
СЕТЕВЫЕ УСТРОЙСТВА И ПРОТОКОЛЫ

Нижний Новгород, 2025г.

МОДЕЛЬ OSI

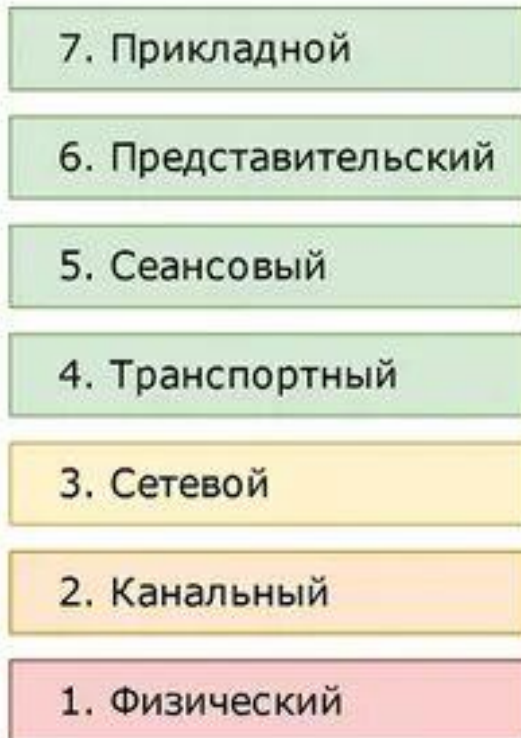


БОКСИНГ СЕТЕВЫХ ПАКЕТОВ

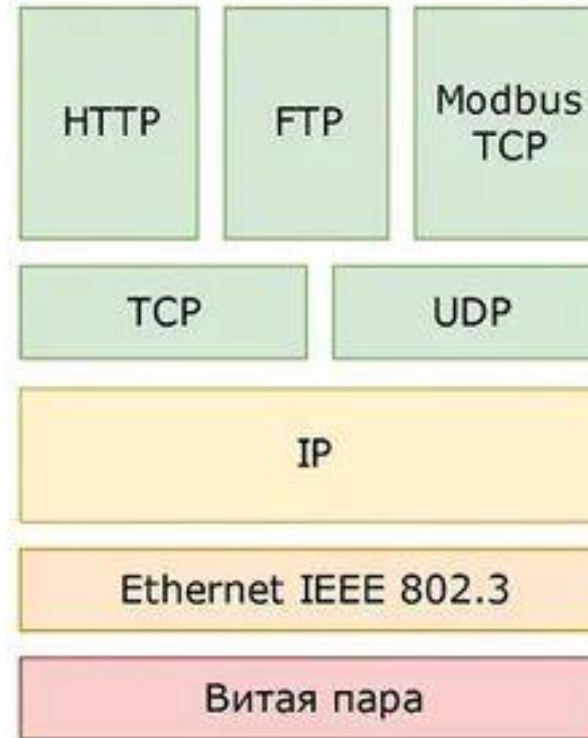


БОКСИНГ СЕТЕВЫХ ПАКЕТОВ

Уровни модели OSI

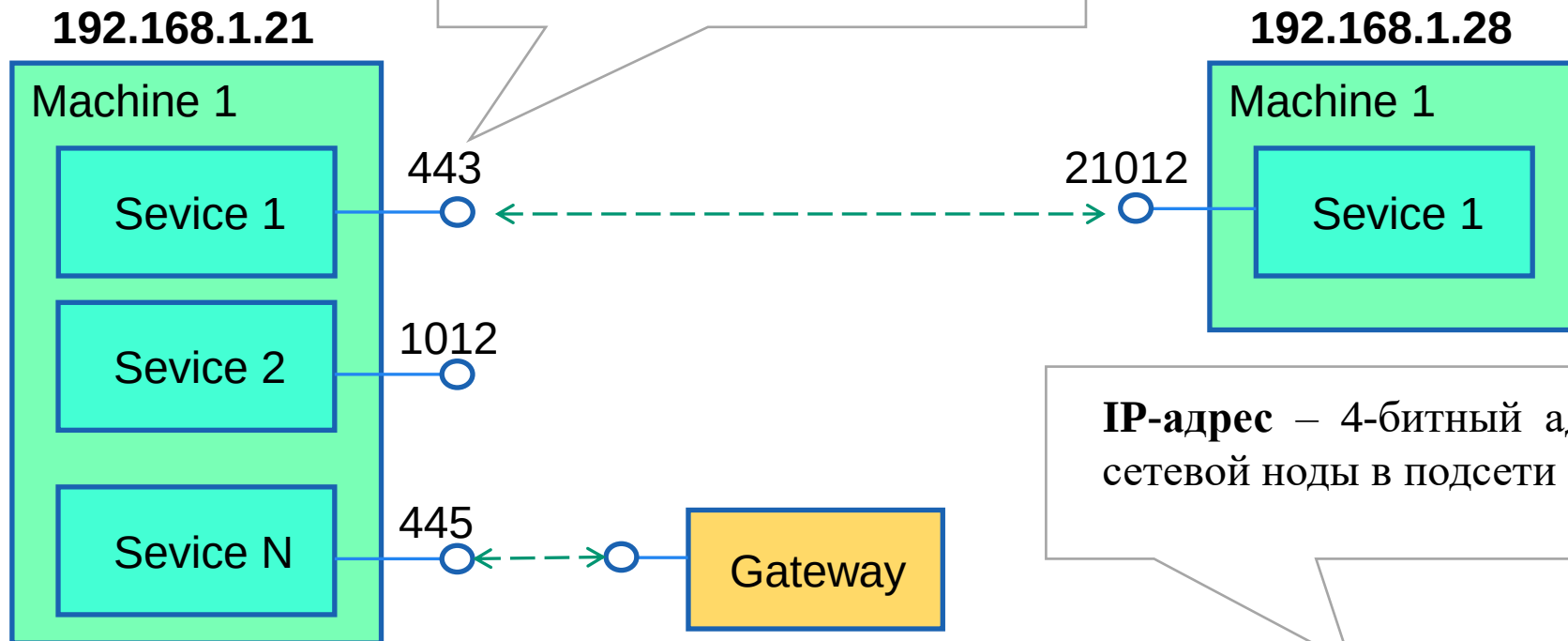


Набор протоколов TCP/IP



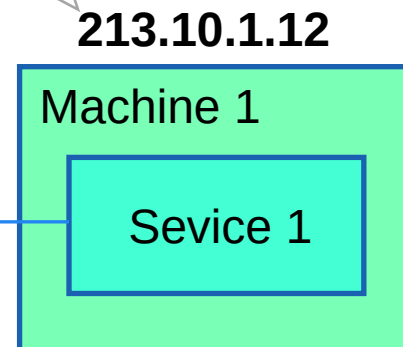
СЕТЕВАЯ АДРЕСАЦИЯ

Порт – ид. номер сервиса на сетевой ноде.



IP-адрес – 4-битный адрес сетевой ноды в подсети

Полный сетевой адрес сервиса
Service 1 на Machine 1:
192.168.1.21 : 443



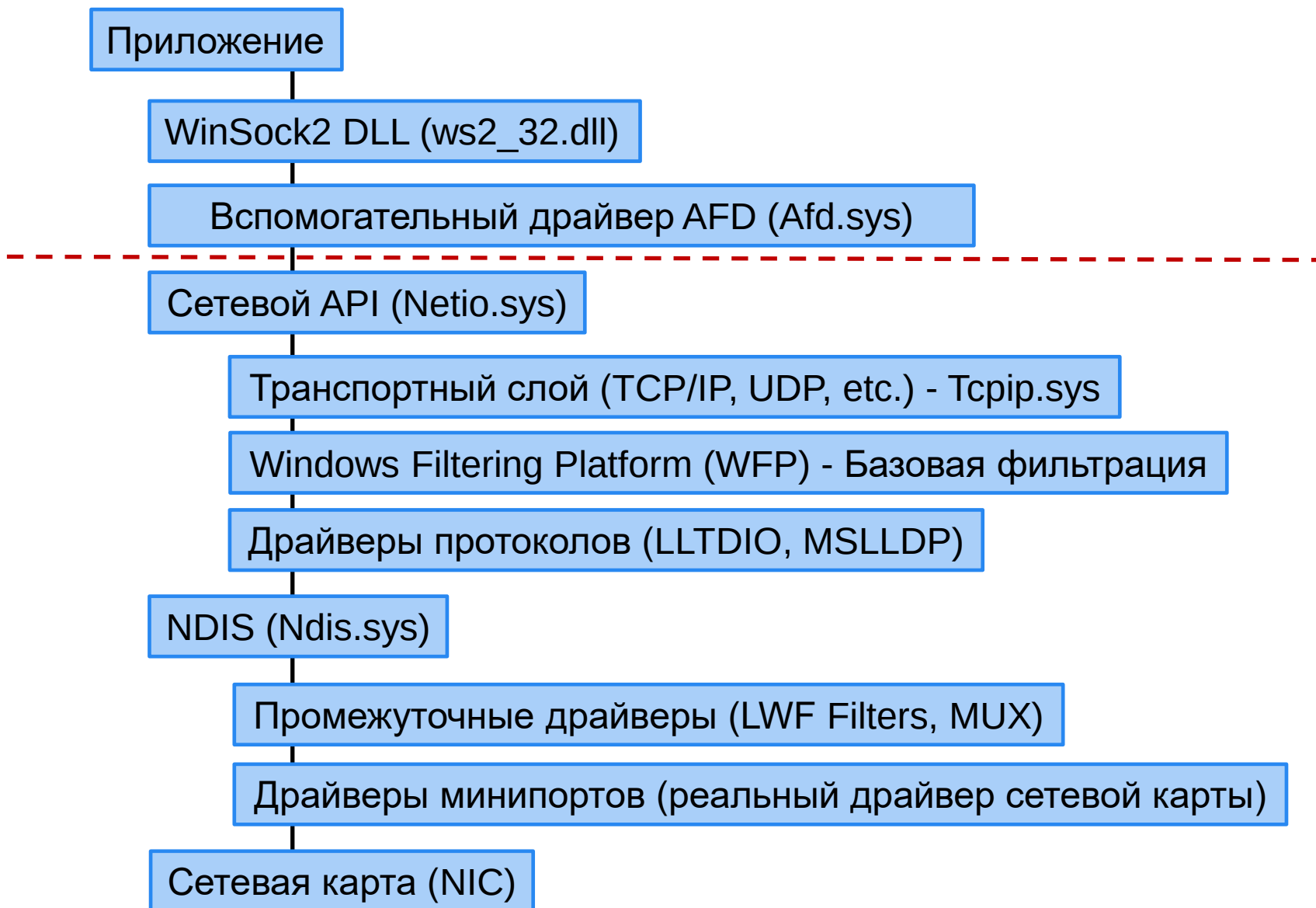
КЛЮЧЕВЫЕ ТРЕБОВАНИЯ

- **Надежность и устойчивость:** Драйвер режима ядра не должен вызывать BSOD.
- **Производительность:** Минимизация задержек, максимизация пропускной способности.
- **Безопасность:** Изоляция, проверка входных данных, соответствие политикам.
- **Расширяемость:** Возможность добавления новых протоколов и фильтров.
- **Совместимость:** Соответствие стандартам (RFC, IEEE) и обратная совместимость API.

КЛЮЧЕВЫЕ ПРИНЦИПЫ

- **Слоистая архитектура:** Четкое разделение на NDIS, транспорт, WFP, WinSock.
- **Безопасность и стабильность превыше гибкости.**
- **Расширяемость через четко определенные интерфейсы** (WFP, NDIS, WinSock SPI), а не через хаки.
- **Критическая важность модели памяти и асинхронного ввода-вывода.**

КЛЮЧЕВЫЕ ТРЕБОВАНИЯ



КЛЮЧЕВЫЕ ТРЕБОВАНИЯ

Пользовательский режим: Изоляция процессов, безопасность. Все вызовы в ядро - через **системные вызовы** (переход через syscall/int 2Eh).

Режим ядра: Привилегированный доступ к аппаратуре и памяти. Общее адресное пространство для всех драйверов.

!!! Важность для сетей: Обработка каждого пакета может требовать нескольких переходов через границу (дорого!). Задача системного ПО - минимизировать эти переходы.

ВЫБОР УРОВНЯ РЕАЛИЗАЦИИ

В режиме ядра (TDI Legacy / WFP Callout / NDIS Протокол):

Плюсы: Макс. производительность, прямой доступ к стеку.

Минусы: Сложность, риск паники, отладка.

Когда: Высокопроизводительный транспорт, глубокая интеграция в стек.

В пользовательском режиме (WinSock SPI / TAP-драйвер / WinDivert):

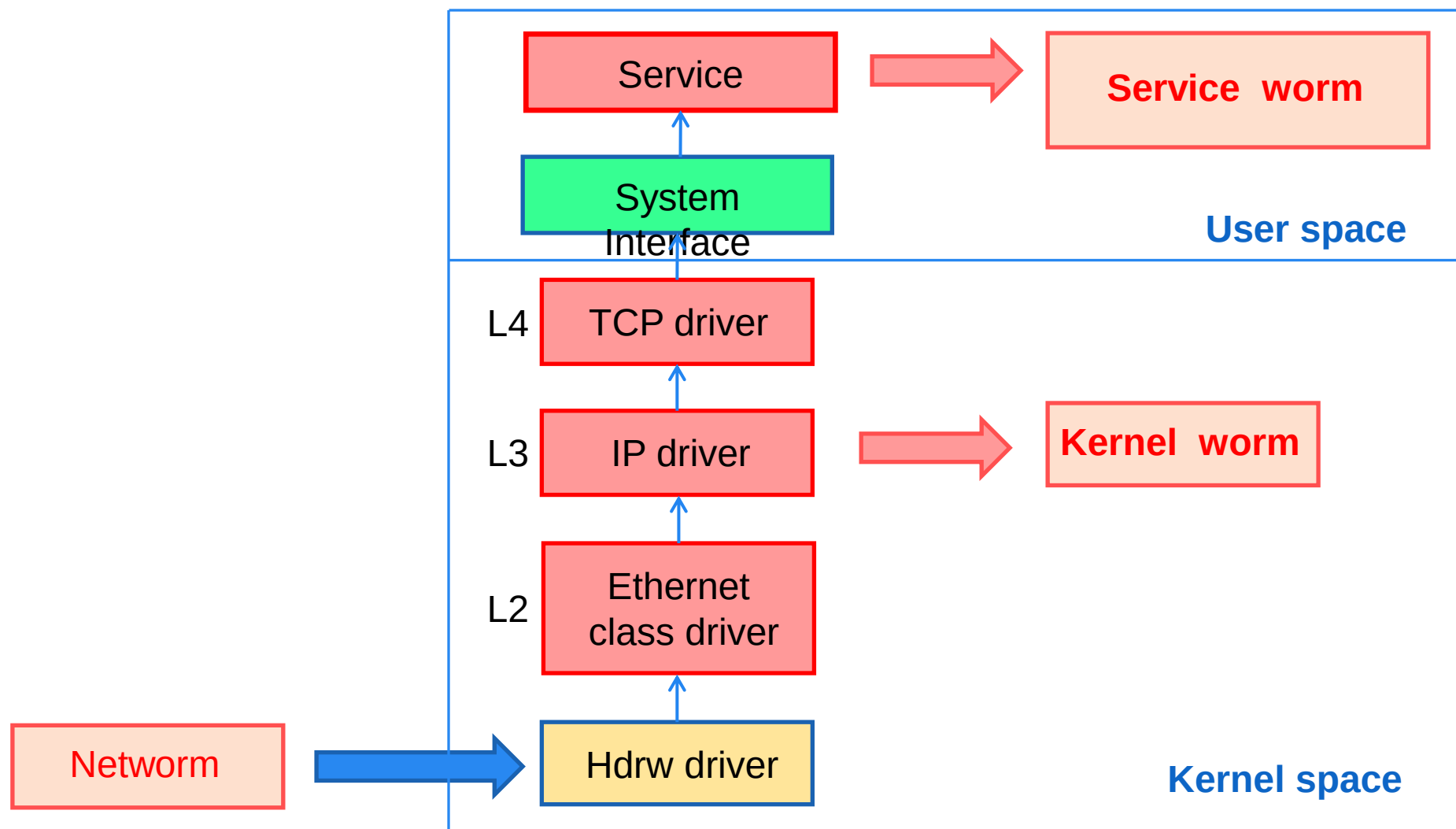
Плюсы: Безопасность, простота разработки.

Минусы: Накладные расходы на переключения, копирование.

Когда: Эксперименты, прокси, VPN-клиенты (частично), эмуляция устройств.

БЕЗОПАСНОСТЬ ДРАЙВЕРА

Инфектор (infector) — часть червя, осуществляющая формирование транспортировочного модуля червя и реализацию атаки.



БЕЗОПАСНОСТЬ ДРАЙВЕРА

WinSock - это не просто библиотека, это **архитектурный компонент**, обеспечивающий абстракцию над сетевыми возможностями ядра.

WinSock Service Provider Interface (SPI): Позволяет встраивать собственные транспорты (например, протокол поверх USB). Это пример расширяемости системного ПО.

Работа с ядром: Вызовы WinSock в конечном счете попадают в **Afd.sys** (Ancillary Function Driver), который взаимодействует с Tcpip.sys.

ЯДРО СЕТЕВОГО СТЕКА

Tcpip.sys

- Монолитный драйвер, реализующий стек TCP/IPv4/v6 (IP, ICMP, IGMP, TCP, UDP)
- Управляет таблицами маршрутизации, ARP-кэшем, сокетами ядра

Netio.sys

- Общий модуль для вспомогательных сетевых операций (создание NET_BUFFER, работа с интерфейсами)
- Предоставляет API для других драйверов протоколов

Важно: Это закрытый, нерасширяемый код Microsoft. Расширение функциональности происходит **вокруг** него (через WFP, драйверы).

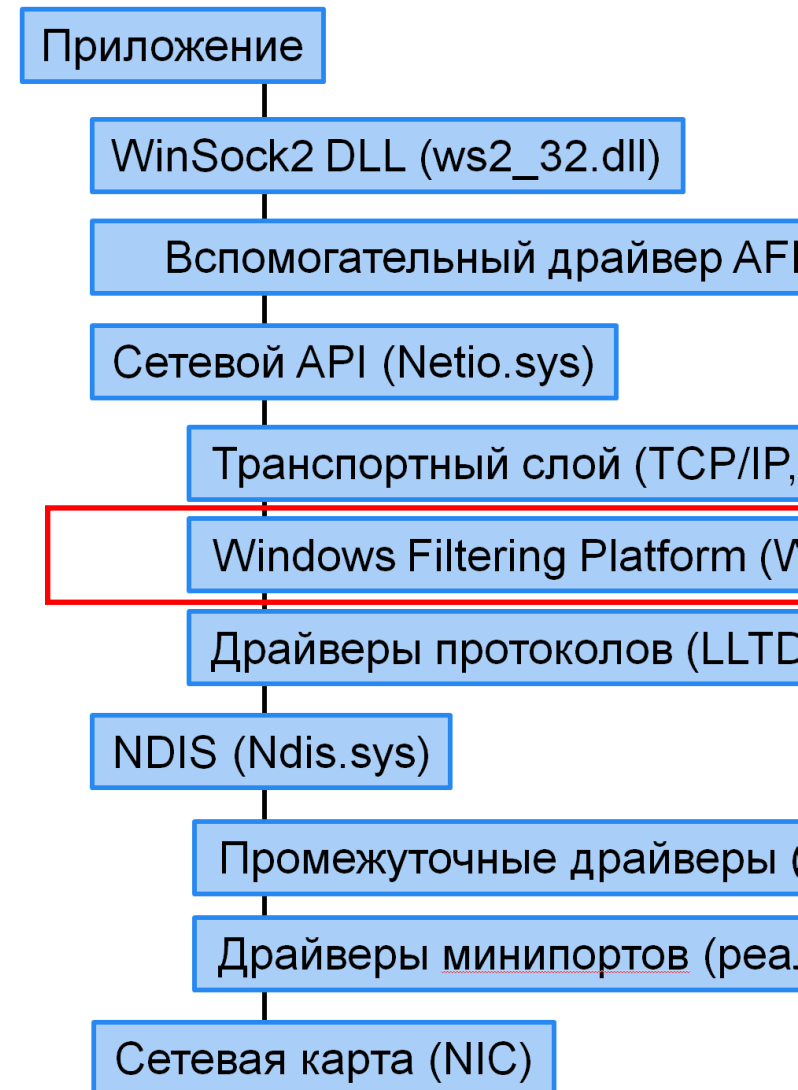
WINDOWS FILTERING PLATFORM (WFP)

Философия: Предоставить безопасный, структурированный API для внедрения логики фильтрации и модификации в произвольную точку стека

Компоненты:

- **Слои (Layers):** Точки принятия решений в стеке (например, FWPM_LAYER_INBOUND_TRANSPORT_V4).
- **Фильтры (Filters):** Правила, применяемые на слоях (разрешить/запретить/вызвать коллаут).
- **Коллауты (Callouts):** Драйверы, реализующие произвольную логику (шифрование, инспекция, модификация)

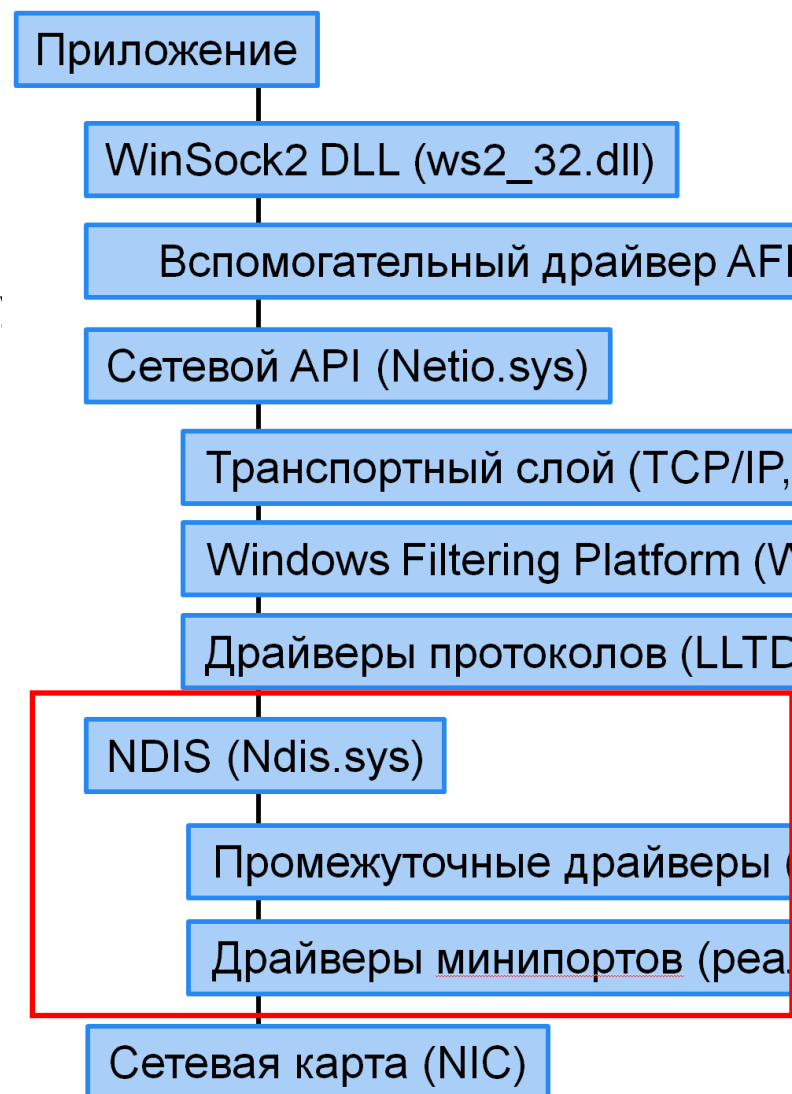
Системная значимость: Замена неструктурированных и опасных хуков ядра (IoAttachDevice). Все современные файрволы и антивирусы используют WFP.



NDIS (NETWORK DRIVER INTERFACE SPECIFICATION)

NDIS.sys: Библиотека режима ядра, абстрагирующая аппаратуру. Реализует общие задачи (очереди, синхронизация).

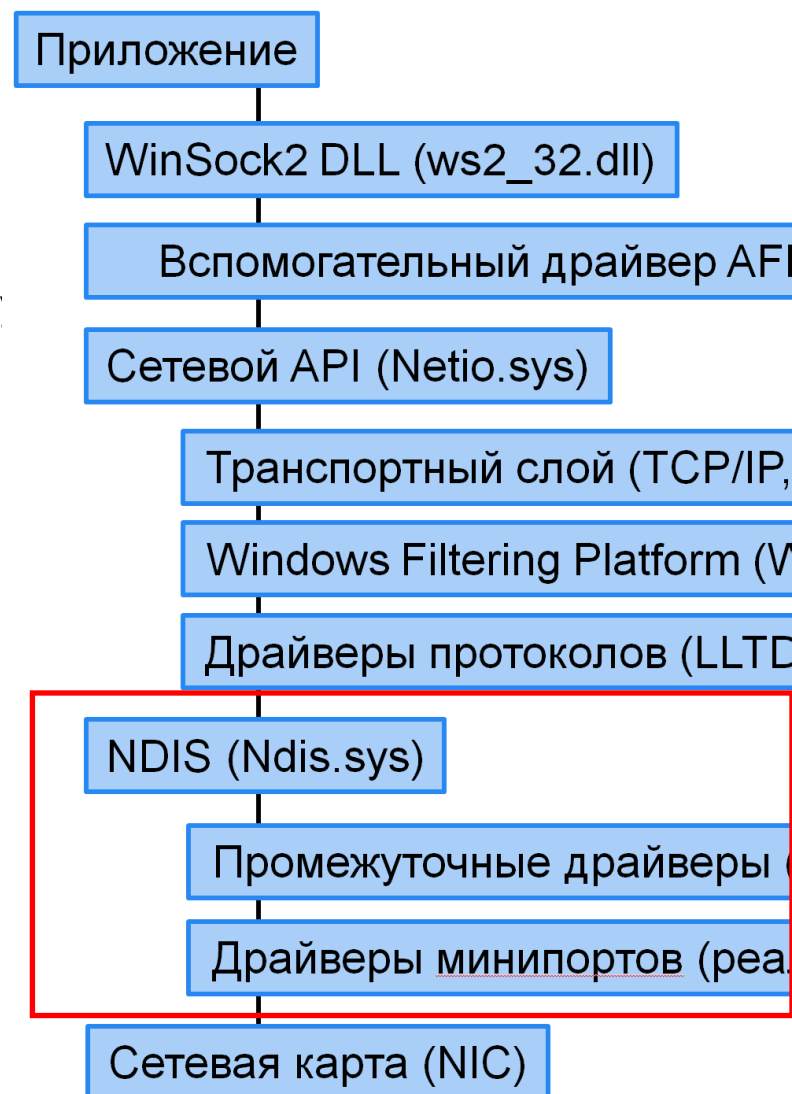
- **Минипорт-драйвер:** Реализует интерфейс к конкретному NIC. Вызывается NDIS для инициализации, отправки, настройки.
- **Промежуточный драйвер (Lightweight Filter - LWF):** "Прозрачный" драйвер между минипортом и протоколом. Для мониторинга, шифрования, виртуализации (Hyper-V Switch).
- **Протокольный драйвер:** Верхний драйвер, который "представляет" протокол стеку (например, драйвер для протокола XYZ).



NDIS (NETWORK DRIVER INTERFACE SPECIFICATION)

NDIS.sys: Библиотека режима ядра, абстрагирующая аппаратуру. Реализует общие задачи (очереди, синхронизация).

- **Минипорт-драйвер:** Реализует интерфейс к конкретному NIC. Вызывается NDIS для инициализации, отправки, настройки.
- **Промежуточный драйвер (Lightweight Filter - LWF):** "Прозрачный" драйвер между минипортом и протоколом. Для мониторинга, шифрования, виртуализации (Hyper-V Switch).
- **Протокольный драйвер:** Верхний драйвер, который "представляет" протокол стеку (например, драйвер для протокола XYZ).



ВЫБОР УРОВНЯ РЕАЛИЗАЦИИ

В режиме ядра (TDI Legacy / WFP Callout / NDIS Протокол):

Плюсы: Макс. производительность, прямой доступ к стеку.

Минусы: Сложность, риск паники, отладка.

Когда: Высокопроизводительный транспорт, глубокая интеграция в стек.

В пользовательском режиме (WinSock SPI / TAP-драйвер / WinDivert):

Плюсы: Безопасность, простота разработки.

Минусы: Накладные расходы на переключения, копирование.

Когда: Эксперименты, прокси, VPN-клиенты (частично), эмуляция устройств.

СОЗДАНИЕ ПРОТОКОЛЬНОГО ДРАЙВЕРА NDIS

Цель: Добавить в систему поддержку нового канального или сетевого протокола (например, протокол для научной аппаратуры).

Шаги:

1. Создать проект KMDF-драйвера с поддержкой NDIS протокола.
2. Зарегистрировать драйвер в DriverEntry (NdisRegisterProtocolDriver).
3. Реализовать callback-функции
(ProtocolBindAdapterEx, ProtocolReceiveNetBufferLists).
4. Связаться с минипортом (NdisOpenAdapterEx).
5. Обработать отправку/прием через NdisSendNetBufferLists / указывая в ProtocolReceiveNetBufferLists.

Итог: Система "видит" новый протокол, может привязывать к нему интерфейсы.

РЕАЛИЗАЦИЯ ЧЕРЕЗ WFP CALLOUT

Цель: Добавить функциональность (например, логирование, блокировку, туннелирование) к существующим протоколам (TCP/IP).

Шаги:

1. Создать KMDF/LEGACY драйвер.
2. Зарегистрировать коллаут и его функции (classifyFn, notifyFn) в WFP.
3. Добавить фильтры, которые будут активировать коллаут на нужных слоях.
4. В classifyFn: анализировать пакет (доступен в NET_BUFFER), принимать решение (FWP_ACTION_PERMIT/BLOCK/CONTINUE), возможно модифицировать данные (инжекция).

Итог: Мощный и безопасный способ внедрения в стек без написания полноценного протокола.

ДРАЙВЕР ВИРТУАЛЬНОГО АДАПТЕРА (TAP-WINDOWS)

Назначение: Создать виртуальный сетевой интерфейс, пакеты с которого попадают в пользовательское приложение (например, для VPN OpenVPN).

Архитектура:

- **Драйвер режима ядра (tap0901.sys):** NDIS минипорт-драйвер, создающий виртуальное устройство.
- **Пользовательская сторона:** Драйвер создает символьное устройство (\Device\Tap...). Приложение открывает его (CreateFile) и читает/пишет IRP (пакеты) через ReadFile/WriteFile.

Системный аспект: Пример разделения функций: критичная для производительности часть (передача пакетов) - в ядре, гибкая логика (шифрование) - в пользовательском процессе.